

Major Project Assignment

Real-Time Emotion Detection (Computer Vision)

Project Overview

Human emotions play a crucial role in communication, decision-making, and user experience. As artificial intelligence continues to evolve, **emotion recognition from visual data** has become an essential capability in fields such as **human-computer interaction, healthcare, education, marketing, and security**.

This project focuses on developing a **Real-Time Emotion Detection System** using **Computer Vision (CV)** and **Deep Learning** techniques. The goal is to capture live webcam footage, detect faces in real time, and classify facial expressions into emotion categories such as **happy, sad, angry, surprised, disgusted, fearful, or neutral**.

The system will use **Convolutional Neural Networks (CNNs)** to learn features from facial images and classify emotions accurately. By integrating face detection with real-time video processing, the project aims to build an interactive application that displays emotion labels directly on the user's screen as they are detected — demonstrating the power of **AI-driven visual perception**.

Objectives

The primary objectives of this project are:

1. Implement real-time face detection from webcam video streams.
2. Train and deploy a **CNN-based emotion classification model**.
3. Recognize and classify human facial expressions into predefined categories.
4. Overlay detected emotions as text labels on the live video feed.
5. (Optional) Extend the system to perform emotion analytics or personalized feedback based on detected emotions.

Suggested Dataset

Use the following publicly available datasets from Kaggle for model training and evaluation:

-  **FER-2013 Facial Expression Dataset:**
<https://www.kaggle.com/datasets/msambare/fer2013>
(A large dataset of labeled facial images across multiple emotion categories.)
-  **Facial Expression Recognition Dataset:**
<https://www.kaggle.com/datasets/jonathanoheix/face-expression-recognition-dataset>
(Useful for training CNN models on a wide range of emotional states.)

Technical Requirements

- Python 3.x
- Deep Learning Framework: **TensorFlow** / **PyTorch**
- Computer Vision: **OpenCV**, **dlib** (optional)
- Data Handling: **NumPy**, **Pandas**
- Visualization: **Matplotlib**, **Seaborn**
- Deployment (Optional): **Streamlit** / **Flask**

Project Workflow & Deliverables

The project is structured into phases to guide the complete development lifecycle.

Phase 1: Data Preparation & Exploration

Task 1: Import and explore the facial emotion dataset. Perform basic data analysis to understand class distribution and image characteristics.

Task 2: Preprocess the dataset:

- Convert images to grayscale (optional but recommended).
 - Resize images to a fixed dimension suitable for CNN input.
 - Normalize pixel values and create training/validation/test splits.
-

Phase 2: CNN Model Development

Task 3: Design and build a **Convolutional Neural Network (CNN)** architecture for emotion classification.

Task 4: Train the CNN on the prepared dataset and evaluate performance using metrics such as accuracy, precision, recall, and F1-score.

Task 5: Implement techniques like data augmentation, dropout, or transfer learning to improve model accuracy.

Phase 3: Real-Time Face Detection Integration

Task 6: Integrate a face detection module using **OpenCV (Haar cascades or DNN)** to locate faces in live webcam video streams.

Task 7: Crop detected face regions and pass them through the trained CNN model for emotion prediction.

Phase 4: Real-Time Emotion Classification

Task 8: Display real-time emotion predictions as text labels directly on the webcam feed. Each detected face should have an associated label (e.g., "Happy", "Sad", "Angry").

Task 9: Optimize the system for real-time inference, ensuring minimal latency between capture and prediction.

Phase 5: Extended Functionality (Optional)

Task 10: Enhance the system with one or more advanced features:

- Emotion analytics dashboard showing emotion distribution over time.
 - Logging emotion data for further analysis.
 - Voice feedback or alerts based on detected emotions.
-



Submission Guidelines

Students must submit the following deliverables:

1. **Code Implementation:**
 - A complete Jupyter Notebook or Python script implementing data preprocessing, model training, and real-time detection.
 2. **Project Report (2–4 pages):**
 - Detailed explanation of methodology, architecture, results, and conclusions.
 3. **Demo Application (Optional):**
 - A real-time emotion detection app built with OpenCV, Streamlit, or Flask.
 4. **Sample Output:**
 - Screenshots or short video showing emotion detection in action.
-



Project Outcomes

Upon completion of this project, students will be able to build a **real-time computer vision application** capable of detecting human emotions with high accuracy. The project demonstrates proficiency in **deep learning, CNNs, real-time video processing, and AI deployment** — skills highly valued in domains like **AI-based user experience, human-computer interaction, healthcare, and surveillance systems**.

